ATS Developer Guide

Day 19 – ATS Documentation & Knowledge Transfer

1. Purpose

This guide provides practical information for developers maintaining, testing, troubleshooting, and extending the ATS implementation.

It complements the ATS Technical Documentation and Architecture Diagrams by focusing on development operations.

---

2. Development Environment

The ATS system is implemented primarily in Python.

The development environment uses a Python virtual environment to isolate project dependencies.

Typical project execution is performed from the project root directory.

Example:

```
C:\Users\Assna VS\zecpath-ai-system
```

Activate the existing virtual environment before running project commands.

Example:

```
.\venv\Scripts\Activate.ps1
```

After activation, the terminal should display:

```
(venv)
```

---

3. Dependency Installation

Dependencies should be maintained through the project's existing requirements configuration.

Typical installation:

```
pip install -r requirements.txt
```

Important libraries used by ATS-related functionality include technologies for:

- PDF processing
- DOCX processing
- Data processing
- Machine learning
- Sentence embeddings
- FastAPI-based API integration
- Testing

Dependency versions should be reviewed before production deployment.

---

4. Important ATS Areas

The ATS implementation is organized into logical areas including:

```
parsers/
scoring/
ats_engine/
utils/
tests/
data/
docs/
```

Each directory should maintain its existing responsibility.

Avoid moving working modules or creating duplicate implementations unless an architectural change has been intentionally approved.

---

5. Parser Maintenance

Parser-related components handle:

- PDF extraction
- DOCX extraction
- Text cleaning
- Resume normalization
- Resume section classification
- Structured resume processing

When modifying parser logic, test with different resume formatting styles.

Important cases include:

- Multiple pages
- Different bullet types
- Excessive whitespace
- Missing sections
- Unusual heading names
- Empty fields

---

6. Resume Section Classifier

The classifier uses normalized heading recognition.

Current standardized sections include:

```
profile
skills
work_experience
education
certifications
projects
```

When adding a new heading variant, map it to an existing logical section where appropriate.

For example:

```
Technical Competencies
```

can map to:

```
skills
```

Avoid creating unnecessary new section categories when an existing category accurately represents the content.

---

7. PROFILE Handling

Resume header information before the first recognized heading is currently treated as:

```
profile
```

This behavior is intentional.

It prevents candidate header information such as:

```
Name
Designation
Contact Header
```

from being automatically classified as UNKNOWN.

When changing section-classification behavior, preserve or deliberately reevaluate this requirement.

---

8. Semantic Matching Engine

Semantic matching uses:

```
all-MiniLM-L6-v2
```

through SentenceTransformer.

The engine calculates similarity for:

- Skills
- Experience
- Projects

The overall semantic similarity is calculated from these component similarities.

---

9. Shared Semantic Model

The semantic model is reused across Semantic Matching Engine instances within the same Python process.

Developers should avoid changing this back to unconditional model initialization inside every engine instance unless there is a specific requirement.

Repeated model loading can introduce unnecessary startup overhead.

---

10. Batch Semantic Encoding

Overall semantic matching encodes:

```
Resume Skills
JD Skills
Resume Experience
JD Experience
Resume Projects
JD Projects
```

together.

When extending semantic matching, prefer batching compatible text inputs rather than introducing unnecessary individual model calls.

---

11. Semantic Regression Test

After changing semantic matching logic, run:

```
python -m tests.test_semantic_matching_engine
```

The existing controlled regression test previously produced:

```
skills_similarity: 76.63
experience_similarity: 66.39
projects_similarity: 84.05
overall_similarity: 75.69
match_level: Good Match
```

The test should complete with:

```
All Semantic Matching tests passed successfully!
```

If expected outputs are intentionally changed, update tests only after confirming that the new behavior is correct.

---

12. ATS Scoring Engine

The ATS Scoring Engine combines:

```
skill_match
experience_relevance
```

```
education_alignment
semantic_similarity
```

using role-specific weights.

The general formula is:

```
ATS Score =
(skill_match × skill_weight)
+
(experience_relevance × experience_weight)
+
(education_alignment × education_weight)
+
(semantic_similarity × semantic_weight)
```

---

13. Role Weight Configuration

Role weights are stored in the ATS weight configuration.

Current tested roles include:

```
Data Scientist
Frontend Developer
Backend Developer
Business Analyst
```

When adding another role:

1. Add the role to the existing weight configuration.
2. Define all required scoring weights.
3. Ensure the weights follow the intended scoring design.
4. Add representative test cases.
5. Run ATS scoring tests.
6. Run system evaluation where appropriate.

Do not silently fall back to unrelated role weights.

---

14. ATS Scoring Test

After modifying scoring logic or role weights, run:

```
python -m tests.test_ats_scoring_engine
```

A successful test should complete without assertion or configuration errors.

---

15. Candidate Ranking

The ranking system sorts candidates by ATS score and assigns rank.

When modifying ranking logic, verify:

- Highest score receives the highest rank position.
- Candidate order remains deterministic where required.
- Required candidate information remains in the output.
- Shortlisting continues to consume the expected structure.

Run:

```
python -m tests.test_candidate_ranking
```

after relevant changes.

---

16. Shortlisting

The tested shortlisting workflow uses:

```
Shortlist Threshold: 80
Review Threshold: 60
```

producing:

```
Shortlisted
Review
Rejected
```

Threshold changes should be configuration-driven where possible and validated against representative candidate data.

---

17. Fairness Engine

The fairness-related implementation includes:

- Resume normalization
- Personal-attribute masking
- Score normalization controls
- Keyword dependency controls
- Semantic contribution indicators
- Bias flags

After modifying fairness-related behavior, run:

```
python -m tests.test_fairness_engine
```

---

18. API Layer

The ATS API exposes system functionality through FastAPI.

The tested API includes support for:

- Health checks
- Resume parsing
- ATS scoring
- Candidate shortlisting
- Validation handling
- Error handling
- Job handling

Run the API test using:

```
python -m tests.test_ats_api
```

The successful Day 16 test validated the API integration flow.

---

19. API Contract Maintenance

When changing an endpoint:

1. Review request fields.
2. Review response fields.
3. Preserve standardized success/error structure.
4. Update validation rules.
5. Update tests.
6. Update API documentation.

7. Verify downstream compatibility.

Avoid changing response structures without reviewing consumers of that endpoint.

---

20. System Evaluation

The ATS system evaluation compares automated decisions against controlled manual-review labels.

Run:

```
python -m tests.test_ats_system_evaluation
```

The Day 17 baseline produced:

```
Accuracy: 91.67%
Precision: 100.00%
Recall: 87.50%
F1 Score: 93.33%
Mismatch Count: 1
```

These values belong to the controlled development dataset.

When changing scoring or thresholds, compare new results against the baseline rather than assuming that a higher or lower individual score automatically represents improvement.

---

21. Performance Testing

After changing:

- PDF extraction
- Text cleaning
- Normalization
- Section classification
- Semantic matching

run:

```
python -m tests.test_ats_performance_optimization
```

The Day 18 measured baseline was:

```
Text cleaning:
0.001457 sec

PDF extraction:
0.105023 sec

Semantic processing:
0.183752 sec

Second semantic engine initialization:
0.001425 sec

Peak tracked Python allocation:
0.0040 MB
```

Timing can vary between runs and machines.

Use repeated controlled benchmarks before claiming percentage performance improvements.

---

22. Recommended Test Order After Major Changes

For ATS changes affecting multiple components, a useful regression order is:

```
Parser / Component Test
        ↓
Semantic Matching Test
        ↓
ATS Scoring Test
        ↓
Ranking Test
        ↓
Fairness Test
        ↓
API Test
        ↓
System Evaluation
        ↓
Performance Test
```

Only the tests relevant to the changed components need to be run for small isolated changes, but broader changes should receive broader regression coverage.

---

23. Logging

System components use logging to record important events and failures.

Logs can help diagnose:

- Parser failures
- Model initialization
- Normalization operations
- Scoring operations
- Section detection
- Unexpected processing errors

Avoid logging:

- Passwords
- API secrets
- Authentication tokens
- Sensitive credentials
- Unnecessary candidate personal information

---

24. Troubleshooting – JSONDecodeError

Possible error:

```
json.decoder.JSONDecodeError:
Expecting value: line 1 column 1
```

Likely causes:

- Empty JSON configuration file
- Invalid JSON syntax
- Missing opening or closing braces
- Trailing or misplaced commas

Resolution:

1. Open the referenced JSON file.
2. Verify that it contains valid JSON.
3. Confirm required configuration objects exist.
4. Save the file.
5. Rerun the relevant test.

---

25. Troubleshooting – Unsupported Job Role

Possible error:

```
Unsupported job role
```

Cause:

The requested role does not exist in the ATS role-weight configuration.

Resolution:

1. Check the exact role name.
2. Review the weight configuration.
3. Add the role only if it is intentionally supported.
4. Define all required weights.
5. Add tests for the new role.

---

26. Troubleshooting – SemanticMatchingEngine ImportError

Possible error:

```
ImportError:
cannot import name 'SemanticMatchingEngine'
```

Check:

```
Get-Content scoring\semantic_matching_engine.py
```

Confirm that the file contains:

```
class SemanticMatchingEngine:
```

A file containing the wrong module implementation will cause the import to fail.

You can verify the import with:

```
python -c "from scoring.semantic_matching_engine import SemanticMatchingEngine;
print('SemanticMatchingEngine import successful')"
```

---

27. Troubleshooting – Hugging Face Warning

A message may appear indicating that requests to Hugging Face Hub are unauthenticated.

This warning does not necessarily indicate that semantic processing failed.

If the model loads and the tests pass, the ATS functionality can still be working.

For environments requiring authenticated Hugging Face access or higher Hub limits, configure authentication according to the deployment environment's approved secret-management process.

Never commit authentication tokens to source control.

---

28. Troubleshooting – API 500 Scoring Error

If the scoring endpoint returns HTTP 500, inspect the structured error message and scoring inputs.

Previous integration issues demonstrated errors such as incompatible input types reaching arithmetic operations.

Check that:

```
skill_match
experience_relevance
education_alignment
semantic_similarity
```

reach the scoring engine as numerical values rather than nested dictionaries or other incompatible objects.

Then rerun:

```
python -m tests.test_ats_api
```

---

29. Troubleshooting – Empty PDF Text

If PDF extraction returns an empty string:

1. Confirm the file path is correct.
2. Confirm the PDF exists.
3. Confirm the PDF contains extractable text.
4. Check parser logs for errors.
5. Test with a known text-based PDF.

Image-only/scanned PDFs may require an OCR-specific workflow, which should be treated as a separate capability if implemented.

---

30. Troubleshooting – Incorrect Resume Sections

If resume content is assigned to the wrong section:

1. Inspect the exact section heading in the resume.
2. Normalize the heading.
3. Check whether it exists in the classifier's supported heading variants.
4. Add an appropriate variant if necessary.
5. Run section-classifier tests.
6. Verify that PROFILE/header behavior remains correct.

---

31. Troubleshooting – UNKNOWN Header Information

Candidate name or designation should not normally become UNKNOWN simply because it appears before the first recognized heading.

Verify that the classifier initializes content under:

```
profile
```

and rerun the Day 18 profile-detection test.

---

32. Troubleshooting – Slow Semantic Matching

If semantic processing becomes unexpectedly slow:

1. Check whether the model is being loaded repeatedly.
2. Confirm shared model reuse is still active.
3. Confirm overall similarity uses batch encoding.
4. Separate cold-start timing from warm-model timing.
5. Test using the Day 18 performance test.
6. Compare results under similar runtime conditions.

---

33. Troubleshooting – Test Failure After Changes

When a previously passing test fails:

1. Read the first meaningful traceback.

2. Identify the module and line causing the failure.
3. Determine whether the input or expected output changed.
4. Fix the implementation rather than deleting the assertion.
5. Rerun the specific failing test.
6. Run related regression tests.
7. Update tests only when an intentional requirement change justifies it.

Do not change expected values simply to make a test pass.

---

34. Adding a New Job Role

To extend ATS scoring to another role:

```
Define Role
    ↓
Configure Weights
    ↓
Create Representative Test Cases
    ↓
Run Scoring Test
    ↓
Run System Evaluation
    ↓
Review Mismatches
    ↓
Document Configuration
```

Role configuration should be based on justified requirements rather than arbitrary values.

---

35. Adding New Resume Section Variants

To support another heading:

1. Identify the logical target section.
2. Add the normalized heading variant.
3. Avoid duplicating equivalent categories.
4. Create a representative resume sample.
5. Run classification tests.
6. Confirm existing headings still work.

---

36. Modifying Scoring Logic

Changes to scoring logic can affect:

- Candidate scores
- Recommendations
- Ranking
- Shortlisting
- Accuracy metrics
- API output

Therefore, scoring changes require broader regression testing than isolated formatting changes.

At minimum, consider rerunning:

```
python -m tests.test_ats_scoring_engine
python -m tests.test_candidate_ranking
python -m tests.test_ats_api
python -m tests.test_ats_system_evaluation
```

depending on the nature of the change.

---

37. Security and Confidentiality

Do not commit or distribute:

- Credentials
- API tokens
- Passwords
- Secret keys
- Confidential PRDs
- Non-public internal diagrams
- Proprietary infrastructure information
- Unapproved candidate data

Use environment-based secret management for credentials where required.

Documentation intended for GitHub or external submission should contain only approved technical information.

---

38. Git Workflow

Before committing changes:

```
git status
```

Review modified files carefully.

Then use:

```
git add .
git commit -m "Describe the implemented change"
git push
```

Before pushing, verify that confidential files, secrets, temporary outputs, or unintended large model/cache files are not staged.

---

39. Maintenance Principles

Future developers should follow these principles:

- Preserve modular responsibilities.
- Avoid duplicate implementations.
- Keep configuration separate where appropriate.
- Maintain explainable scoring outputs.
- Add tests for new behavior.
- Run regression tests after shared-component changes.
- Measure optimization instead of assuming it.
- Track mismatches rather than hiding them.
- Protect sensitive information.
- Update documentation when behavior changes.

---

40. Future Improvement Areas

Recommended future development areas include:

- Larger human-reviewed ATS evaluation datasets
- More non-tech role testing
- Role-specific threshold calibration
- Borderline manual-review zones
- Fresher-aware evaluation
- Expanded entity detection
- Additional document-format robustness
- Production load testing
- Concurrency testing
- Monitoring and observability
- Versioned API contracts
- Automated regression pipelines

---

41. Knowledge Transfer Summary

A developer taking over the ATS should understand four key areas:

Processing

How resumes are extracted, cleaned, normalized, and classified.

Evaluation

How skills, experience, education, and semantic similarity contribute to candidate scoring.

Decision Logic

How scores lead to recommendations, ranking, and shortlisting.

Validation

How tests, performance measurements, mismatch analysis, and fairness controls are used to verify system behavior.

---

42. Conclusion

The ATS has been structured so that its major processing and evaluation stages can be understood, tested, and extended independently.

Developers should preserve the existing modular structure, use configuration for role-specific behavior, maintain explainable scoring, run appropriate regression tests after changes, and document significant modifications.

This guide serves as the practical knowledge-transfer reference for future ATS maintenance and development.